



UNIVERSIDAD CENTROAMERICANA

JOSÉ SIMEÓN CAÑAS

FUNDAMENTOS DE PROGRAMACIÓN

Sección 02

Documentación del proyecto “Sistema de Comida Universitaria”

Estudiantes:

Ayala Menjívar, Juan Pablo	00149726
Cerón Ponzéand, José Antonio	00013826
Navarro Adame, Ricardo Antonio	00173626
Osorio Rivera, Ernesto Josue	00013826

Fecha de entrega del proyecto: Miércoles 08 de julio del 2026.

Descripción general del sistema

El Sistema de Comida Universitaria es un programa que simula el funcionamiento básico de una cafetería en una universidad. El sistema permite al usuario consultar un menú de alimentos y bebidas, seleccionar los productos que desea comprar, modificar su pedido, calcular el total a pagar y cancelar la compra en caso de ser necesario.

El programa, además, verifica que los productos estén disponibles para evitar pedidos mayores al inventario existente y utiliza archivos para guardar la información de las compras realizadas, generando una factura o comprobante. El proyecto fue diseñado con programación estructurada y usando únicamente las herramientas vistas en la materia.

Objetivo del proyecto

- Mejorar la atención al cliente en la cafetería a través de una interfaz de consola simple y eficaz. Se pretende disminuir los errores en las cuentas manuales y simplificar la gestión de los insumos del día a día, asegurando que se registre con precisión cada una de las transacciones realizadas a lo largo de la jornada laboral.
- Desarrollar un sistema interactivo que simula el proceso de compra en una cafetería universitaria, aplicando los conocimientos adquiridos a lo largo del ciclo mediante el uso de tipos de datos, operadores, estructuras condicionales, ciclos, arreglos y manejo de archivos.

Reglas o funcionamiento

El sistema arranca mostrando un menú principal, en el que se muestran todas las opciones que tiene el usuario para elegir. Con este menú se pueden hacer las diversas acciones del programa.

- El usuario podrá ver el menú de alimentos y bebidas disponibles con su precio y cantidad existente.
- Para hacer un pedido el usuario elige el producto y la cantidad que quiere adquirir.
- Antes de incluir el producto en el pedido, el sistema confirma que haya suficiente disponibilidad del mismo.
- El usuario tiene la opción de cambiar las cantidades de los productos elegidos, o bien cancelar la compra completa antes de finalizar.
- El sistema calcula automáticamente el total a pagar y, si se cumple la condición establecida por el programa, procede a aplicar el descuento correspondiente.
- Cuando finaliza la compra, el usuario tiene la opción de guardar una factura en un archivo de texto, en el cual se registran los productos comprados y el total de la compra.
- El programa sigue en ejecución hasta que el usuario seleccione la opción de salir del sistema.

Opciones del menú

Opción 1: Ver menú y agregar productos

Esta opción muestra la interfaz de toma de pedidos para el cliente, mostrando una lista de cada uno de los productos disponibles, con su nombre y su precio unitario:

- Almuerzo Ejecutivo (\$3.25)
- Pupusa (\$0.60)
- Pan con Pollo (\$2.00)
- Bebida / Refresco (\$1.00)

Al elegir una opción válida, solicita la cantidad deseada y la suma al acumulador de la sesión.

Opción 2: Ver detalle del pedido actual

El cliente puede visualizar los productos almacenados en el carrito de compra. Si el número de productos es a cero, muestra de forma desglosada el nombre del producto, las unidades seleccionadas, el subtotal por artículo y el subtotal neto general. Si no hay elementos, informa que el carrito está vacío.

Opción 3: Cancelar compra (vaciar carrito)

Ayuda a borrar el pedido actual que estaba siendo realizado por el cliente y también permite que las selecciones realizadas por el mismo en su proceso de compra se reestablezcan y de ese modo pueda comenzar desde cero si así lo desea.

Opción 4: Proceder al pago y facturar

Es el paso final para dar por finalizada la compra. Cuenta con una opción de poder realizar un descuento por estudiante, imprimir el ticket en la interfaz de consola y guardar el registro de la factura en un archivo de texto.

Explicación del uso de los temas vistos en clase

Primero, nos enfocaremos en las estructuras condicionales **if** y **else**, las cuales evalúan una condición booleana (verdadera o falsa) para decidir qué bloque de código ejecutar. En el programa, estas se utilizan para comprobar que el usuario elija un producto válido (del 1 al 4) y que la cantidad ingresada sea mayor a 0. También actúan como filtros de visualización, permitiendo que en el carrito y en la factura se muestren únicamente los productos cuya cantidad sea mayor a cero. Finalmente, se emplean en el reporte para comparar las ventas una a una y determinar cuál fue la más alta y cuál la más baja del día.

Por otro lado, la estructura **switch** funciona como un selector múltiple que evalúa una variable entera y salta directamente al caso (case) correspondiente. En este sistema se aplica en tres escenarios:

1. **El menú principal:** Dirige al usuario según la opción marcada (1. Agregar, 2. Ver Carrito, 3. Cancelar, 4. Pagar).
2. **La identificación de productos:** Suma la cantidad ingresada exactamente a la variable del producto seleccionado (almuerzo, pupusa, pan o bebida).
3. **El cálculo de descuentos:** Aplica un 10% de descuento si la variable `esEstudiante` es igual a 1.

En cuanto a los bucles, los ciclos **while** y **do-while** repite un bloque de código mientras una condición se mantenga activa:

- **Do-while (garantiza al menos una ejecución):** Envuelve todo el flujo principal del programa. Permite atender al primer cliente y, al final, evalúa si se debe abrir el menú para un nuevo cliente (`otraCompra == 1`).
- **While (evalúa la condición antes de entrar):** Por un lado, mantiene al cliente actual dentro del menú de compras hasta que decide salir presionando la opción 4. Por el otro, se utiliza para validar errores de teclado (`cin.fail()`); si el usuario introduce letras en lugar de números, el ciclo limpia el flujo de entrada y le exige el dato correcto repetidamente.

Luego, los ciclos **for** se usan para recorrer arreglos (*arrays*) cuando ya se conoce el número exacto de iteraciones que se van a realizar. En este código se utiliza para:

- **El historial de ventas:** Recorre todas las ventas guardadas para sumarlas, calcular el promedio y buscar los montos máximos y mínimos.
- **El producto más vendido:** Revisa el registro de los 4 productos disponibles para identificar cuál de ellos tuvo la mayor cantidad de unidades vendidas.

Continuando con los temas, encontramos los **arreglos (arrays)**, que son estructuras de datos utilizadas para almacenar múltiples valores del mismo tipo bajo un solo nombre y en posiciones de memoria contiguas. En este programa se emplean de tres formas clave:

1. **El historial de ventas (historialVentas):** Un arreglo de tipo flotante (`double`) que guarda el monto total pagado por cada cliente de forma individual, permitiendo almacenar hasta un límite definido por la constante `MAX_VENTAS`.

2. **Los catálogos del menú (nombreProductos y preciosProductos):** Dos arreglos paralelos de tamaño 4 que guardan los nombres (cadenas de texto) y los precios de la comida. Al compartir el mismo índice (por ejemplo, la posición 0 corresponde al "Almuerzo Ejecutivo" y a su precio de \$3.25), facilitan la organización y consulta de los datos del menú.
3. **El contador de estadísticas (totalesPorProducto):** Un arreglo temporal de 4 posiciones que se llena al final del día con el acumulado de ventas de cada producto para poder comparar fácilmente cuál fue el más vendido mediante un ciclo.

Por otro lado, se implementan las **funciones**, que son bloques de código independientes diseñados para realizar tareas específicas, lo que ayuda a modularizar el programa, evitar la duplicación de código y mejorar su lectura. El sistema utiliza:

- **El archivo de cabecera (funciones.h):** Donde se definen los prototipos (las firmas o declaraciones) de las funciones y se utiliza la palabra clave extern para que las variables globales del archivo principal puedan ser compartidas y modificadas desde fuera.
- **procesarFacturacion():** Una función encargada de toda la lógica del cierre de cuenta: pregunta si el cliente es estudiante, calcula el descuento, imprime el detalle en pantalla y genera la factura física.
- **guardarVenta():** Una función interna que se encarga exclusivamente de la contabilidad, sumando los productos y el dinero recaudado a las estadísticas globales del día y registrando el pago en el historial.
- **mostrarReporte():** La función final que procesa todos los datos acumulados durante la jornada para calcular e imprimir promedios, montos máximos, mínimos y el producto estrella del día.

Finalmente, el programa incorpora el **manejo de archivos** a través de la librería <fstream> y, específicamente, mediante el objeto ofstream (salida de archivos). Esto permite que los datos no se pierdan al cerrar la consola. En el código, se utiliza para crear un archivo de texto llamado "factura.txt". Antes de escribir en él, el sistema usa la función condicional archivo.is_open() para verificar que el archivo se haya creado correctamente en el disco duro. Una vez asegurado, en lugar de mandar la información a la pantalla con cout, utiliza el flujo archivo << para escribir de

manera permanente el diseño de la factura, los subtotales, el descuento y el total a pagar, cerrando el documento al terminar con `archivo.close()`.

Flujograma digital

Debido a que el tamaño del flujograma no permite que sea mostrado directamente en este documento, se puede consultar en formato digital haciendo clic en el siguiente enlace:

https://viewer.diagrams.net/?tags=%7B%7D&lightbox=1&highlight=0000ff&edit=_blank&layers=1&nav=1&title=anteproyecto%20-%20cafeteria'&dark=0#Uhttps%3A%2F%2Fdrive.google.com%2Fuc%3Fid%3D153vmjBc1N7KF9yDPeBNnvoC6skuz2WXd%26export%3Ddownload

Explicación del manejo de archivos

En el transcurso del desarrollo del proyecto, se pudo identificar que el uso de variables convencionales limitaba la persistencia de los datos ya que al cerrar la consola, inmediatamente se perdía la información de la compra, ya que la memoria RAM es volátil. Para poder solucionar esto y poder simular un entorno de facturación real dentro de la cafetería universitaria, se implemento el manejo de archivos de salida mediante la librería estándar `<fstream>`. Esto permite que el sistema vaya más allá de la ejecución del programa, guardando un comprobante físico en el disco duro.